



كلية العلوم
السملاية - مراكش
FACULTÉ DES SCIENCES
SEMLALIA - MARRAKECH

UNIVERSITE CADI AYYAD

FACULTE DES SCIENCES SEMLALIA - MARRAKECH

Année universitaire 2025/2026

Rapport de Projet ML

Ai Platform – Plateforme pour L'apprentissage supervisée

Module : AI & Machine Learning

Filière : IASD

Encadré Par : Prof. Ameksa Mohammed

Réalisé par :

Bousebbat Kamal

Maarouf Qamar

Belaicha Salma

Table des matières

Introduction	3
Spécification de besoins	4
Environnement et développement	5
Préparation des données	9
L'exploration et l'analyse	10
Modélisation – Algorithme supervisée	11
L'évaluation des modèles	13
Exportation des modèles	15
Intégration avec Django	16
Conclusion	17
Annexes	18

Rapport du projet : Ai Plateforme

Introduction

Ce projet s'inscrit dans le cadre du module de **Machine Learning** et a pour objectif de développer une **plateforme web intelligente**, réalisée avec **Django**, permettant d'entraîner, comparer, sauvegarder et exploiter plusieurs modèles d'apprentissage automatique.

Il s'agit d'un projet complet qui combine **développement web, analyse de données, conception de modèles prédictifs, et intégration d'outils professionnels** tels que Jupyter Notebook, Git et Scikit-learn.

La plateforme repose sur une architecture modulaire permettant aux utilisateurs de charger des données, explorer leur contenu, entraîner différents algorithmes, évaluer leurs performances, puis effectuer des prédictions à travers une interface web simple, intuitive et interactive.

Les modèles intégrés dans ce projet appartiennent à la famille du **Machine Learning supervisé**, notamment :

- Régression Linéaire
- Régression Logistique
- Decision Tree
- Random Forest
- SVM (Support Vector Machine)
- XGBoost

Chaque modèle est entraîné dans **Jupyter Notebook**, analysé statistiquement, puis exporté au format **.pkl** pour être utilisé par le framework Django.

Ce workflow permet d'appliquer de manière concrète l'ensemble du cycle d'un projet d'apprentissage automatique : **préparation des données, normalisation, entraînement, validation, Evaluation et utilisation**.

Le projet a également été développé en utilisant **Git et GitHub** afin d'assurer une bonne organisation du travail, un suivi précis des versions et une collaboration fluide.

L'utilisation du contrôle de version (**Git**) nous a permis de travailler de manière professionnelle, en simulant les conditions d'un projet réel.

Ce travail nous a offert une expérience complète couvrant :

- l'analyse exploratoire et la préparation des données,
- l'entraînement et l'évaluation de plusieurs modèles supervisés,
- la sauvegarde et le chargement de modèles dans une application web,
- l'intégration backend (Django) et frontend (HTML/CSS/Bootstrap),
- la gestion du code et du versioning avec Git,

En somme, ce projet représente une **synthèse pratique** de nos apprentissages en Machine Learning et en développement web.

Il nous a permis de relier la théorie à une application concrète, tout en nous initiant aux bonnes pratiques utilisées dans les projets professionnels : modularité, structuration du code, séparation des responsabilités, documentation et travail collaboratif.

Spécification des besoins

La plateforme IA développée repose sur un ensemble de fonctionnalités visant à permettre l'entraînement, la comparaison et l'utilisation de plusieurs modèles de Machine Learning supervisé. Elle intègre une interface web réalisée avec Django, une partie analytique effectuée dans Jupyter Notebook, ainsi qu'un système de chargement de modèles exportés en .pkl.

Le système se veut simple, intuitif et suffisamment flexible pour accueillir différents algorithmes et jeux de données.

Besoins fonctionnels :

- **Affichage d'une page d'accueil** présentant le projet, les algorithmes utilisés et la navigation principale.
- **Entraînement des modèles** (Régression Linéaire, Logistique, SVM, Decision Tree, Random Forest, XGBoost) via Jupyter Notebook.
- **Exportation des modèles entraînés en fichiers .pkl** afin d'être utilisés sur la plateforme Django.
- **Chargement des modèles dans Django** pour effectuer des prédictions en temps réel.
- **Formulaire utilisateur** permettant de saisir les données nécessaires à la prédiction.
- **Affichage du résultat de prédiction** sur une page dédiée (valeur prédite).
- **Comparaison des performances** des modèles à travers des métriques (Accuracy, F1-score, RMSE, etc.).
- **Navigation claire** entre les pages : Accueil, Algorithmes, À propos, connexion, registrer.
- **Intégration de Git/GitHub** pour la gestion des versions du projet.

Besoins non fonctionnels :

- **Interface intuitive et simple à utiliser.**

La plateforme doit permettre une navigation fluide entre les différentes pages (Accueil, Algorithmes, À propos, connexion, registrer), tout en offrant une expérience utilisateur claire.

- **Code structuré, réutilisable, lisible et bien commenté.**

L'architecture Django doit respecter l'organisation MVC : séparation entre les *views*, les *templates*, les fichiers *static*, et les modèles.

Les notebooks Jupyter doivent être également organisés et documentés afin de faciliter la compréhension des étapes : prétraitement, entraînement, évaluation et exportation du modèle.

- **Compatibilité avec les environnements supportant Python, Django et Scikit-learn.**

Le projet doit fonctionner sur les systèmes d'exploitation modernes (Windows) disposant de Python 3.13 et 3.11, ainsi que les bibliothèques nécessaires (NumPy, Pandas, Scikit-learn, XGBoost, Django).

- **Chargement rapide des modèles et exécution efficace des prédictions.**

Les fichiers .pkl doivent être chargés rapidement au démarrage de l'application, et la prédiction doit s'exécuter sans latence perceptible, afin d'offrir un service performant et réactif.

Environnement de développement

L'environnement de développement constitue la base technique du projet et permet d'assurer une exécution correcte, stable et reproductible des différents modules de Machine Learning ainsi que de l'application web Django. Cette section détaille l'installation des outils nécessaires, l'organisation du projet et la gestion des dépendances, selon les standards professionnels applicables aux projets d'intelligence artificielle.

Installation et configuration :

La première étape consiste à mettre en place les outils indispensables au développement du projet. Celui-ci repose principalement sur l'écosystème Python, Jupyter Notebook et le framework Django.

1- Python

Python constitue le langage principal du projet, notamment pour la manipulation des données, la création des modèles supervisés et l'intégration dans Django.

- Version recommandée : **Python 3.11 et 3.13**
- Vérification de l'installation : **python --version**

2-Jupyter Notebook

Jupyter Notebook est utilisé pour réaliser l'analyse exploratoire des données, le prétraitement, l'entraînement des modèles et leur exportation en fichiers .pkl.

- Installation : **pip install notebook**
- Lancement : **jupyter notebook**

3-Django

Django assure la partie web permettant de charger les modèles entraînés, d'exécuter les prédictions et d'afficher les résultats à l'utilisateur.

- Installation : **pip install django**
- Création du projet : **django-admin startproject aiPlatform**

4-Bibliothèques Machine Learning

Les principales librairies utilisées dans le projet sont :

- Scikit-learn : **pip install scikit-learn**
- XGBoost : **pip install xgboost**
- Pandas, NumPy : traitement et transformation des données → **pip install pandas numpy**

Outils de versioning : Git et GitHub

Git est utilisé pour la gestion des versions, permettant une collaboration efficace et une traçabilité de toutes les modifications apportées au projet.

Création et gestion des branches

Le travail a été organisé à travers plusieurs branches afin de séparer le développement des différentes fonctionnalités.

Vérifier dans quel branch on se trouve, le branche par défaut et (main)

git branch

Créer et passer directement à une branche :

git checkout -b nom_de_branche

Cette commande crée la branche et l'active automatiquement, ce qui facilite le développement de fonctionnalités indépendantes.

Ajout des modifications

Une fois les fichiers modifiés ou ajoutés, ils sont indexés via :

git add.

Validation (Commit)

Après l'ajout, les modifications sont enregistrées avec un message décrivant le changement :

git commit -m "Description des modifications"

Un message clair et explicite permet de mieux comprendre l'évolution du projet.

Envoi vers GitHub (Push)

Les modifications validées sont ensuite envoyées vers le dépôt distant :

git push origin nom_de_branche

Récupération des mises à jour (Pull)

Pour synchroniser le projet local avec la dernière version du dépôt distant :

git pull origin nom_de_branche

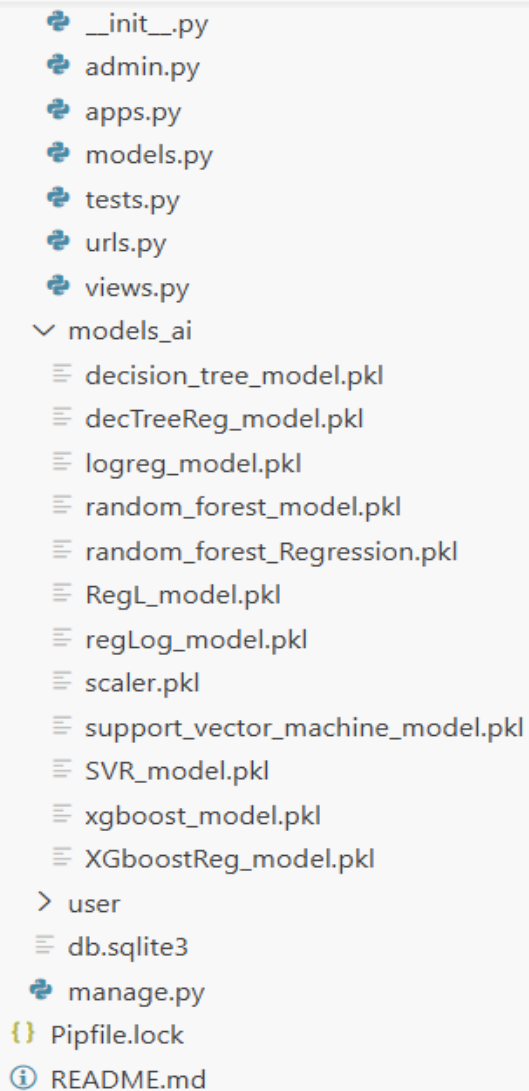
Cette commande permet de récupérer les nouvelles modifications et éviter les conflits.

Organisation du Projet :

Une organisation claire et structurée assure une meilleure lisibilité du code, une maintenance facilitée et un déploiement sans erreur. La plateforme est divisée en plusieurs modules correspondant à des tâches spécifiques : entraînement des modèles données, interface web et scripts de prédiction.

Structure générale de recommandée

```
▼ ML_PROJECT
  ▼ aiPlateform
    ▼ aiPlateform
      > __pycache__
      📄 __init__.py
      📄 asgi.py
      📄 settings.py
      📄 urls.py
      📄 wsgi.py
    ▼ algoAi
      > __pycache__
      > migrations
      > static
      ▼ templates
        > Decision_tree
        > DecTree_Reg
        ▼ includes
          <> footer.html
          <> header.html
        > randforest_details
        > random_forest_regression
        > Regression_linear
        > Regression_Logistique
        > Support_Vector_Machine
        > SVM_Regression
        > XGboost_classification
        > XGboost_Regression
        <> about.html
        <> accueil.html
        <> index.html
```



```
__init__.py
admin.py
apps.py
models.py
tests.py
urls.py
views.py
▼ models_ai
  decision_tree_model.pkl
  decTreeReg_model.pkl
  logreg_model.pkl
  random_forest_model.pkl
  random_forest_Regression.pkl
  RegL_model.pkl
  regLog_model.pkl
  scaler.pkl
  support_vector_machine_model.pkl
  SVR_model.pkl
  xgboost_model.pkl
  XGboostReg_model.pkl
  > user
    db.sqlite3
  manage.py
  {} Pipfile.lock
  ⓘ README.md
```

Cette structure respecte les bonnes pratiques de développement : séparation claire entre les modèles ML, les notebooks, les fichiers Django, les données et les ressources statiques.

Gestion des Dépendances :

Afin de garantir que le projet fonctionne correctement sur différents environnements, la gestion des dépendances est effectuée à travers un environnement virtuel Python.

Environnement virtuel (Virtualenv)

Création d'un environnement isolé : `python -m venv env`

Activation de l'environnement **Windows** : `env\Scripts\activate`

L'utilisation d'un environnement virtuel combiné à un fichier de dépendances assure une reproductibilité totale du projet, essentielle dans le contexte du développement d'applications de Machine Learning.

Préparation des données

La **préparation des données** est une étape cruciale dans tout projet de Machine Learning, car la qualité des données influence directement la performance des modèles. Cette phase comprend plusieurs sous-étapes :

Importation et exploration initiale des données :

Les données ont été importées à l'aide de la bibliothèque pandas de Python. Une première exploration a permis de vérifier la structure du jeu de données, le type des variables et de détecter d'éventuelles valeurs manquantes ou aberrantes.

Nettoyage des données :

Gestion des valeurs manquantes : Les colonnes avec des valeurs manquantes ont été soit complétées par la moyenne/mediane/mode, soit supprimées si le pourcentage de valeurs manquantes était trop élevé, dans notre deux jeux de données il n'y a pas de valeurs manquantes.

Correction des valeurs aberrantes : Les données incohérentes (outliers) ont été détectées à l'aide de méthodes statistiques (écart interquartile ou Z-score) ici on a utilisé IQR et traitées selon le contexte.

Transformation des variables :

- **Encodage des variables catégorielles :** Les variables non numériques ont été converties en format numérique à l'aide de techniques telles que l'encodage One-Hot ou Label Encoding.
- **Normalisation / Standardisation :** Les variables numériques ont été mises à l'échelle afin de faciliter la convergence des algorithmes de Machine Learning (ex : StandardScaler)

Séparation des jeux de données :

Le jeu de données a été divisé en **ensemble d'entraînement** et **ensemble de test** pour évaluer la performance du modèle de manière fiable. La séparation typique utilisée est de 80% pour l'entraînement et 20% pour le test.

Vérification finale :

Après ces transformations, le jeu de données était prêt pour l'étape d'entraînement. Une dernière vérification a été réalisée pour confirmer l'absence de valeurs manquantes, d'outliers.

L'exploration et l'analyse

L'exploration et l'analyse des données constituent une étape essentielle permettant de comprendre la structure du jeu de données, d'identifier ses caractéristiques principales et de guider le choix des techniques de prétraitement et des modèles.

Analyse descriptive

Une première analyse descriptive a été effectuée afin d'obtenir une vue d'ensemble sur les variables :

- Vérification du nombre d'observations et des types de variables (numériques et catégorielles).
- Calcul des statistiques descriptives : moyenne, médiane, variance ...
- Détection d'éventuels déséquilibres dans les classes cible (pour un problème de classification).

Cette étape a permis d'identifier les variables les plus influentes ainsi que les relations initiales entre elles.

Identification des anomalies

L'analyse exploratoire a révélé :

- Absence des valeurs manquantes.
- Présence de valeurs aberrantes dans le jeu de données réservé pour la régression et absence des outliers dans le jeu de données réservé pour la classification.
- Equilibrage de la variable cible (LeaveOrNot) du Dataset de la classification avec SMOTE et from scratch (manuellement).

Encodage et normalisation des variables

Suite à l'analyse exploratoire, certaines transformations ont été appliquées afin de préparer les données pour l'entraînement des modèles :

- **Encodage des variables catégorielles :**
Les variables catégorielles ont été converties en format numérique à l'aide de deux techniques :
 - *Label Encoding* pour les variables ordinales ou binaires.
 - *One-Hot Encoding* pour les variables nominales afin d'éviter l'introduction d'un ordre artificiel.
- **Standardisation des variables numériques :**
Les attributs continus ont été standardisés à l'aide du **StandardScaler**, permettant de ramener les valeurs à une distribution centrée et réduite.
Cette étape favorise la convergence et améliore les performances des modèles sensibles à l'échelle des données (ex : régression linéaire, SVM).
-
- Après ce processus, on passe à la division des données.

Modélisation – Algorithme supervisée

Dans le cadre de ce projet, plusieurs algorithmes de Machine Learning supervisé ont été testés afin d'identifier celui offrant les meilleures performances pour chaque type de problème (régression et classification).

Les modèles sélectionnés couvrent des approches linéaires, arborescentes, à marge maximale ainsi que des méthodes d'ensemble avancées.

Régression linéaire

Principe

La régression linéaire cherche à modéliser la relation entre une variable cible continue et un ensemble de variables explicatives en ajustant une droite minimisant l'erreur quadratique.

Elle repose sur l'équation :

$$\hat{y} = ax + b$$

Avantages

- Simple à comprendre et à interpréter.
- Très rapide à entraîner.
- Fonctionne bien lorsque les variables sont linéairement corrélées à la cible.

Limites

- Ne capture pas les relations non linéaires.
- Sensible aux outliers.
- Performances limitées si les données ne suivent pas une distribution proche du linéaire.

Régression Logistique

Principe

La régression logistique modélise la probabilité qu'une observation appartienne à une classe (0 ou 1).

Elle utilise la fonction sigmoïde :

$$P(y = 1 | x) = \frac{1}{1 + e^{-(z)}}$$

Avantages

- Très performante dans les problèmes de classification binaire.
- Offre des probabilités comme sortie.
- Facile à interpréter.

Limites

- Ne capture pas la non-linéarité sans transformation des variables.
- Mauvaise performance en présence de variables très corrélées.

Decision Tree (Arbre de décision)

Principe

Un arbre de décision segmente l'espace des données en posant des questions successives (seuils). Il construit une structure en forme d'arbre où chaque nœud représente une condition, et chaque feuille une prédiction.

Avantages

- Interprétation très intuitive.
- Capture les relations non linéaires.
- Fonctionne sans normalisation des données.

Limites

- Très sensible au surapprentissage (overfitting).
- Peut devenir instable selon les données.

Random Forest

Principe

Random Forest est un ensemble d'arbres de décision. Il construit plusieurs arbres indépendants et combine leurs prédictions.

Avantages

- Réduit fortement l'overfitting par rapport à un seul arbre.
- Très robuste aux outliers et aux données bruitées.
- Fonctionne bien dans la plupart des situations.

Limites

- Moins interprétable qu'un arbre simple.
- Peut devenir lourd avec beaucoup d'arbres.

SVM (Support Vector Machine)

Principe

SVM cherche à trouver l'hyperplan qui sépare au mieux les classes, en maximisant la marge entre elles. Grâce aux **kernels** (*Linear*, *RBF*...), il peut gérer des frontières de décision non linéaires.

Avantages

- Très efficace pour les données de petite et moyenne taille.
- Excellent pour des frontières non linéaires grâce au kernel RBF.
- Très robuste, bon généraliseur.

Limites

- Sensible au choix des hyperparamètres (C, gamma).
- Peu efficace sur les très grands datasets.
- Ne donne pas directement des probabilités.

XGBoost

Principe

XGBoost (Extreme Gradient Boosting) est un algorithme puissant basé sur les **arbres de décision**

boosting :

il entraîne des arbres successifs où chaque arbre corrige les erreurs du précédent.

Il optimise une fonction de perte grâce au Gradient Descent.

Avantages

- Généralement l'un des modèles les plus performants en pratique.
- Très efficace pour les relations non linéaires.
- Inclut la régularisation, ce qui limite l'overfitting.
- Rapide, optimisé, supporte le parallélisme.

Limites

- Sensible au tuning des hyperparamètres.
- Moins interprétable que des modèles simples.
- Peut-être coûteux en ressources pour les très grands datasets.

L'évaluation des modèles

L'évaluation des modèles constitue une étape fondamentale permettant de mesurer la qualité des prédictions et de comparer les performances des différents algorithmes utilisés. Cette phase permet de sélectionner le modèle le plus adapté au problème traité (régression ou classification) en se basant sur des métriques objectives et fiables.

Métriques d'évaluation pour la régression

Pour le problème de régression, plusieurs indicateurs ont été utilisés afin d'évaluer l'écart entre les valeurs réelles et les valeurs prédites :

- **MSE (Mean Squared Error)**
Mesure l'erreur quadratique moyenne. Plus la valeur est faible, meilleure est la précision du modèle.
- **RMSE (Root Mean Squared Error)**
Racine du MSE, exprimée dans la même unité que la variable cible, ce qui facilite l'interprétation.

- **MAE (Mean Absolute Error)**
Mesure l'erreur absolue moyenne. Moins sensible aux valeurs aberrantes que le MSE.
- **R² Score (Coefficient de détermination)**
Indique la proportion de la variance expliquée par le modèle. Plus R² est proche de 1, meilleure est la performance.

Métriques d'évaluation pour la classification

Pour le problème de classification binaire, l'évaluation repose sur plusieurs mesures clés dérivées de la matrice de confusion :

- **Accuracy score**
Mesure la proportion de prédictions correctes sur l'ensemble du dataset.
- **Recall (Taux de rappel)**
Capacité du modèle à identifier correctement les cas positifs. Important lorsqu'il est essentiel de minimiser les faux négatifs.
- **Precision**
Mesure la qualité des prédictions positives (niveau de confiance dans la classe "1").
- **F1-Score**
Moyenne harmonique entre la précision et le recall. Très utile en cas de léger déséquilibre entre les classes.

Comparaison et sélection du modèle final

Après l'analyse des différentes métriques, les modèles ont été comparés afin de sélectionner celui offrant les meilleures performances pour chaque tâche.

Le choix final s'est basé sur :

- La précision globale
- La stabilité des résultats
- La capacité du modèle à généraliser sur les données de test
- La cohérence des performances entre les différents indicateurs

Exportation des modèles

Après l'entraînement et l'évaluation des différents algorithmes de Machine Learning, les modèles sélectionnés ont été exportés afin d'être réutilisés facilement sans devoir les réentraîner.

Cette étape permet d'intégrer les modèles dans d'autres applications et garantit la reproductibilité des résultats.

Objectif de l'exportation

L'exportation vise à :

- Sauvegarder les modèles finaux après entraînement
- Rendre les prédictions accessibles dans des environnements externes
- Eviter le coût computationnel d'un réentraînement
- Assurer la cohérence entre les résultats obtenus durant les tests et ceux produits en production.

Méthode d'exportation

Les modèles ont été enregistrés sous forme de fichiers grâce au module **joblib**, qui permet de sérialiser les objets Python de manière efficace.

Cette méthode permet de sauvegarder :

- Le modèle entraîné
- Les hyperparamètres optimaux
- Les transformations utilisées (StandardScaler, LabelEncoder...)

Structure des fichiers exportés

Chaque modèle a été exporté dans un fichier distinct pour faciliter son utilisation.

Exemples de fichiers générés :

- decision_tree_model.pkl
- decTreeReg_model.pkl
- logreg_model.pkl
- random_forest_model.pkl
- random_forest_Regression.pkl
- RegL_model.pkl
- Support_vector_machine_model.pkl
- SVR_model.pkl
- Xgboost_model.pkl
- scaler.pkl

Cette organisation permet de charger individuellement chaque composant selon les besoins de l'application.

Chargement et utilisation des modèles

Une fois exportés, les modèles peuvent être rechargés à tout moment dans un script Python ou une interface utilisateur pour réaliser des prédictions, grâce à une simple instruction de désérialisation.

Cette étape garantit :

- Une réutilisation fiable des modèles
- Une facilité d'intégration dans un système de prédiction réel

Intégration avec Django

L'intégration du modèle de Machine Learning dans Django permet de créer une application web interactive où l'utilisateur peut soumettre des données et obtenir une prédiction en temps réel. Cette étape consiste à connecter le modèle entraîné avec l'interface web, la logique backend, ainsi qu'éventuellement la base de données.

Chargement du modèle

Le modèle entraîné (fichier .pkl) est chargé dans les vues Django. Cela permet d'utiliser le modèle déjà entraîné sans devoir le réentraîner à chaque requête. Ce chargement se fait généralement avec joblib.

Création des vues (views.py)

Dans Django, les vues sont responsables de :

- Recevoir les données entrées par l'utilisateur via un formulaire,
- Les préparer (prétraitement, normalisation... si nécessaire),
- Appeler le modèle pour générer une prédiction,
- Renvoyer le résultat à l'interface utilisateur.

Une vue typique reçoit des valeurs depuis une requête POST, puis retourne la prédiction dans une page HTML.

Formulaire de saisie

L'utilisateur accède à une page permettant d'entrer les paramètres nécessaires pour la prédiction (ex. âge, poids...).

Les données envoyées par ce formulaire seront traitées dans la vue du modèle.

Routage (urls.py)

Chaque fonctionnalité est accessible via un lien défini dans le fichier urls.py.

On définit des routes pour :

- Afficher le formulaire
- Lancer la prédiction
- Montrer le résultat

Interface utilisateur (templates)

Les pages HTML affichent :

- Le formulaire pour saisir les données,
- Le résultat de la prédiction,
- Détails concernant l'algorithme

Exécution du modèle dans Django

Le workflow complet est le suivant :

1. L'utilisateur remplit un formulaire.
2. Django reçoit les données via une requête POST.
3. Les données sont prétraitées comme dans la phase d'entraînement.
4. Le modèle fait une prédiction.
5. Le résultat est renvoyé à la page HTML.

Conclusion

Le projet réalisé a permis de **déployer un modèle de Machine Learning supervisé** au sein d'une application web Django, offrant ainsi une démonstration concrète de l'intégration de **l'intelligence artificielle** dans un environnement interactif et accessible aux utilisateurs.

Cette intégration a plusieurs impacts significatifs :

Tout d'abord, l'utilisateur **bénéficie d'une interface intuitive** lui permettant de saisir des données de manière simple et rapide, et d'obtenir des prédictions en temps réel. Cette réactivité est rendue possible grâce à l'utilisation de modèles pré-entraînés (**.pkl**), qui évitent le besoin de réentraînement lors de chaque requête et garantissent une expérience fluide et efficace.

Ensuite, le **backend Django joue un rôle crucial dans la gestion des requêtes utilisateurs**, le prétraitement des données et l'exécution sécurisée des modèles. La structure claire de l'application, organisée via les fichiers **urls.py** et **views.py**, facilite non seulement la maintenance mais aussi l'évolution future du projet. Cela permet, par exemple, d'intégrer facilement de nouveaux modèles, d'ajouter des fonctionnalités avancées.

Par ailleurs, ce projet illustre l'importance de combiner des compétences en **développement web** et en **intelligence artificielle**. Il démontre comment des technologies apparemment distinctes peuvent être orchestrées pour créer des solutions interactives, pratiques et adaptées aux besoins réels des utilisateurs.

Enfin, ce projet ne se limite pas à la simple mise en œuvre d'un modèle prédictif, il représente une **véritable passerelle** entre **l'intelligence artificielle** et **l'expérience utilisateur**, démontrant le potentiel d'applications web interactives qui exploitent pleinement les capacités des technologies modernes.

Annexes

Code Django pour l'intégration des modèles de Machine Learning

Vues Django (views.py)

Le fichier views.py contient toutes les fonctions qui gèrent l'affichage des pages et l'exécution des modèles ML. Chaque vue suit généralement ce workflow :

1. Recevoir les données du formulaire via request.POST.
2. Prétraiter les données (encodage, normalisation, transformation...).
3. Charger le modèle ML correspondant.
4. Effectuer la prédiction.
5. Préparer le contexte (context) pour le template HTML.
6. Renvoyer la page HTML avec le résultat.

Exemples de vues principales :

```
from django.shortcuts import render
import os,joblib
import numpy as np
from django.contrib.auth.decorators import login_required

# Create your views here.

def accueil(request):
    return render(request,'accueil.html')

@login_required(login_url='/user/login')
def index(request):
    return render(request,'index.html')

def about(request):
    return render(request,'about.html')

#Regression Logistique
def regLog_details(request):
    return render(request,'Regression_Logistique/regLog_details.html')

def regLog_atelier(request):
    return render(request,'Regression_Logistique/regLog_atelier.html')

def regLog_tester(request):
    return render(request , 'Regression_Logistique/vehicles_form.html')
```

```

def load_models(name):
    # Remonte d'un niveau dans la structure des dossiers (pour aller de 'app/views.py'
    à 'app/')
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

    # Construit le chemin vers le dossier contenant les modèles (e.g., 'app/models_ai')
    models_dir = os.path.join(base_dir, 'models_ai')

    # Construit le chemin complet du fichier du modèle (e.g.,
    'app/models_ai/logreg_model.pkl')
    model_path = os.path.join(models_dir, name)

    # Charge le modèle sérialisé à l'aide de la librairie joblib
    ml_model = joblib.load(model_path)

    return ml_model

def reglog_prediction(request):
    #Tâche 1 : Recevoir le Colis
    if request.method == 'POST':
        # Tâche 2 : Déballer le Colis
        education = request.POST.get('Education')
        joining_year = int(request.POST.get('JoiningYear'))
        city = request.POST.get('City')
        payment_tier = int(request.POST.get('PaymentTier'))
        age = int(request.POST.get('Age'))
        gender = request.POST.get('Gender')
        ever_benched = request.POST.get('EverBenched') # "Yes" ou "No"
        experience = float(request.POST.get('ExperienceInCurrentDomain'))

        if(education.lower()=='Bachelors'):
            education_model=0
        elif(education.lower()=='Masters'):
            education_model=1
        else:
            education_model=2

        City_Pune=0
        City_New_Delhi=0
        City_Bangalore=0
        if(city.lower()=='bangalore'):
            City_Bangalore=1
        elif(city.lower()=='new delhi'):
            City_New_Delhi=1
        else:
            City_Pune=0

```

```

# Convertir EverBenched en binaire si nécessaire
ever_benched_model = 1 if ever_benched.lower() == 'yes' else 0
gender_model = 1 if gender.lower() == 'male' else 0

# Tâche 3 : Réveiller l'Expert
# cette fonction (load_models) est défini avant
model = load_models('regLog_model.pkl')

# Tâche 4 : Poser la Question à l'Expert
prediction =
model.predict([[education_model,joining_year,payment_tier,age,gender_model,ever_benched
_model,experience,city_Bangalore,city_New_Delhi,city_Pune]])
predicted_class = prediction[0]

# Debug : afficher la prédiction dans la console
print("=== DEBUG ===")
print("Input DataFrame :")
print(predicted_class)
print("Predicted Class :", predicted_class)
print("=====")

# Tâche 5 : Traduire la Réponse
employee_leave = {0: 'NOT LEAVING', 1: 'LEAVING'}
img_url = {'NOT LEAVING':'images/random_forest1.jpeg',
'LEAVING':'images/decTree2.jpg'}
pred_vehicule = employee_leave[predicted_class]
pred_img = img_url[pred_vehicule]

# Tâche 6 : Préparer le Plateau-Repas (context)
input_data = {
    'education':education,
    'joining_year':joining_year,
    'city':city,
    'payment_tier':payment_tier,
    'age':age,
    'gender':gender,
    'ever_benched':ever_benched,
    'experience':experience
}

context = {
    'leaving':pred_vehicule,
    'img_emp':pred_img,

```

```

        'inital_data' : input_data # NOTE: Il y a une faute de frappe dans l'image
        ('inital_data' au lieu de 'initial_data')
    }

    return render(request, 'Regression_Logistique/regLog_results.html', context)

    return render(request, 'Regression_Logistique/vehicles_form.html')

```

Les autres vues suivent une structure similaire : **Régression Linéaire, Decision Tree, Random Forest, SVM, XGBoost.**

```
@login_required(login_url='/user/login')
```

@login_required est un **décorateur Django** qui protège une vue :
il oblige l'utilisateur à être connecté pour accéder à la page.

Routage (urls.py) → Exemple de Code

Le fichier urls.py relie les URLs aux vues correspondantes :

```

from django.urls import path
from . import views

# app_name = "algoAi"

urlpatterns = [

    path('', views.accueil, name='accueil'),
    path('algorithmes', views.index, name='index'),
    path('about', views.about, name='about'),
    path('reglog_details/', views.regLog_details, name='reglog_details'),
    path('reglog_atelier/', views.regLog_atelier, name='reglog_atelier'),
    path('reglog_tester/', views.regLog_tester, name='reglog_tester'),
    path('reglog_prediction/', views.reglog_prediction, name='reglog_prediction'),
]

```

Chaque URL correspond à une **vue spécifique**. Les noms (name) sont utilisés dans les templates pour créer des liens facilement avec {% url 'name' %}.

Remarques pour le lecteur

- Les fonctions load_models() permettent de **charger les modèles ML déjà entraînés**.
- Le code contient des traitements spécifiques aux modèles :

- **Encodage manuel des variables catégorielles** (Education, City, Gender, etc.)
- **Normalisation/Scaler** pour certaines features numériques.
- Les templates HTML (regLog_results.html, Reg_Linear_results.html...) affichent les résultats des prédictions et les données saisies

Authentification :(folder user)

Urls.py

```
from django.urls import path
from . import views

app_name = 'user'

urlpatterns = [
    path('register', views.user_register , name='register'),
    path('login', views.user_login , name='login'),
    path('logout', views.user_logout , name='logout'),
]
```

Explication

- `app_name = 'user'`
→ Permet de regrouper les URLs de cette application et d'éviter les conflits entre noms d'URL dans tout le projet Django.
- `urlpatterns`
→ Liste qui contient les routes (URLs) de l'application.
- `path('register', views.user_register, name='register')`
→ Associe l'URL **/register** à la vue `user_register`, qui gère l'inscription des utilisateurs.
- `path('login', views.user_login, name='login')`
→ Associe l'URL **/login** à la vue `user_login`, qui gère l'authentification.
- `path('logout', views.user_logout, name='logout')`
→ Associe l'URL **/logout** à la vue `user_logout`, qui gère la déconnexion.

Résumé

Ce fichier organise la navigation liée aux utilisateurs. Chaque URL permet d'accéder à une fonctionnalité précise : inscription, connexion ou déconnexion. Cela rend l'application plus structurée et facilite l'intégration avec les vues et les templates.

Views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth.forms import UserCreationForm , AuthenticationForm
from django.contrib.auth import login , logout
```

```
# Create your views here.

def user_register(request):
    if request.method == "POST":
        form = UserCreationForm(request.POST)
        if form.is_valid():
            login(request, form.save())
            return redirect("index")
    else:
        form = UserCreationForm()
    return render(request, 'register.html', {"form": form})

def user_login(request):
    if request.method == "POST" :
        form = AuthenticationForm(data = request.POST)
        if form.is_valid():
            login(request, form.get_user())
            if 'next' in request.POST :
                return redirect(request.POST.get("next"))
            else :
                return redirect("index")
    else :
        form = AuthenticationForm()
    return render(request, 'login.html', {"form": form})

def user_logout(request):
    if request.method == "POST" :
        logout(request)
        return redirect("index")
    else :
        form = AuthenticationForm()
    return render(request, 'logout.html', {"form": form})
```

Explication

Ce fichier contient les fonctions qui gèrent l'inscription, la connexion et la déconnexion des utilisateurs dans l'application Django. Il utilise les formulaires intégrés de Django pour simplifier l'authentification.

User_register(request)

Gère l'inscription d'un nouvel utilisateur.

- Si la méthode est **POST**, Django récupère les données envoyées par le formulaire `UserCreationForm`.

- Si le formulaire est valide, l'utilisateur est créé puis automatiquement connecté avec `login()`.
- L'utilisateur est ensuite redirigé vers la page principale `index`.
- Si la méthode est **GET**, un formulaire vide est affiché.

But : permettre à un utilisateur de créer un compte.

User_login(request)

Gère la connexion d'un utilisateur existant.

- Si la méthode est **POST**, Django utilise `AuthenticationForm` pour vérifier le nom d'utilisateur et le mot de passe.
- Si les identifiants sont corrects, l'utilisateur est connecté via `login()`.
- Si un paramètre `next` existe (page protégée visitée avant connexion), Django redirige vers celle-ci.
Sinon, il renvoie vers `index`.
- En cas de **GET**, le formulaire de connexion est affiché.

But : authentifier un utilisateur et gérer les redirections.

User_logout(request)

Gère la déconnexion.

- Si la méthode est **POST**, Django déconnecte l'utilisateur avec `logout()` puis le redirige vers `index`.
- Sinon, une page de confirmation est affichée (via `logout.html`).

But : permettre à l'utilisateur de se déconnecter en toute sécurité.

Utilisation dans `header.html`

```
{% if user.is_authenticated %}
    <a href="{% url 'index' %}">
        <span role="img" aria-label="new-post" title="new-
post"></span>
    </a>
    <form action="{% url 'user:logout' %}" method="post"
class="logout">
        {% csrf_token %}
        <button class="btn btn btn-outline-danger m-1" aria-
label="User Logout" title= "User Logout">
            logout
        </button>
    </form>
{% else %}
    <li class="nav-item">
```

```

                                <a class="btn btn-primary m-1" href="{% url 'user:login'
%}">Login</a>
                                </li>
                                <li class="nav-item">
                                    <a class="btn btn btn-outline-light m-1" href="{% url
'user:register' %}">Register</a>
                                </li>
                                {% endif %}

```

Explication :

{% if user.is_authenticated %}

Vérifie si l'utilisateur est connecté.

- **Si l'utilisateur est connecté :**
 - Affiche un lien vers la page d'accueil (index).
 - Affiche un **formulaire de déconnexion** contenant un bouton « logout ».
- Le formulaire utilise :
- `method="post"` : sécurité pour les actions sensibles.
 - `{% csrf_token %}` : protection contre les attaques CSRF.

{% else %}

Si l'utilisateur n'est pas connecté :

- Affiche un bouton **Login** (renvoie vers `user:login`)
- Affiche un bouton **Register** (renvoie vers `user:register`)

{% endif %}

Termine la condition.

Résumé simple

Ce code gère dynamiquement l'affichage du menu selon l'état de l'utilisateur :

- **Connecté** → bouton Logout + lien vers l'accueil.
- **Non connecté** → boutons Login et Register.